Задача 1.17: решение уравнения $1 + e^{-2x^2} = \alpha^2 e^{-2x}$ методом простой итерации

Подробный разбор: математика и код

Содержание

1 Что сделано и как запустить **1**

2 Анализ уравнения **2**

2.1 Случай $\alpha = 0$ 2

2.2 Случай $\alpha \neq 0$: переход к логарифмам 2

2.3 Существование и единственность — прямое доказательство 2

3 Метод простой итерации **3**

3.1 Идея 3

3.2 Выбор отображения G 3

3.3 Доказательство сжимаемости 3

3.4 Критерий остановки 4

3.5 Почему число итераций разное 4

3.6 Вычислительные тонкости 4

4 Разбор кода **5**

5 Проверка результатов **7**

6 Возможные вопросы на сдаче **7**

1 Что сделано и как запустить

Файлы в папке `task117_yarik/`:

- `iterative_solver.cpp` — программа (устроена так же, как образец `iterative_solver.cpp` из корня репозитория);
- `report.pdf` (`report.tex`) — отчёт в формате образца `report.pdf`;
- `razbor.pdf` (`razbor.tex`) — этот разбор.

Сборка и запуск (нужен только компилятор C++, сторонние библиотеки не нужны):

```
g++ -O2 -Wall -std=c++17 -o iterative_solver iterative_solver.cpp
./iterative_solver 2.5        # одно значение alpha (любое вещественное)
./iterative_solver -10 10     # все целые alpha от -10 до 10
```

Вывод — таблица из столбцов α, x (найденное решение), n (номер приближения, удовлетворяющего точности 10^{-12}).

Суть решения в пяти строках

1. При $\alpha = 0$ решений нет (левая часть ≥ 1, правая $= 0$).
2. При $\alpha \neq 0$ логарифмируем: $2x + \ln(1 + e^{-2x^2}) = 2\ln|\alpha|$.
3. Выражаем x: $x = G(x) = \ln|\alpha| - \frac{1}{2}\ln(1 + e^{-2x^2})$.
4. $|G'(x)| = \dfrac{2|x|}{e^{2x^2} + 1} \leq \dfrac{1}{2}$ на всей прямой $\Rightarrow$ G — сжатие с $k = \frac{1}{2}$ $\Rightarrow$ (Банах) решение существует, единственно, итерации сходятся.
5. Останавливаемся, когда $\dfrac{|x_{n+1} - x_n|}{1 - k} \leq \varepsilon$, тогда $|x_n - x^*| \leq \varepsilon$.

2 Анализ уравнения

2.1 Случай $\alpha = 0$

Левая часть $1 + e^{-2x^2} > 1$, а правая равна 0. Решений нет. Программа в этом случае печатает `no solution`.

2.2 Случай $\alpha \neq 0$: переход к логарифмам

Обе части уравнения строго положительны, поэтому логарифмирование — равносильное преобразование:

$$\ln\bigl(1 + e^{-2x^2}\bigr) = \ln\alpha^2 - 2x \iff F(x) := 2x + \ln\bigl(1 + e^{-2x^2}\bigr) = 2\ln|\alpha|.$$

Симметрия. В уравнение входит только α^2, поэтому ответы при α и $-\alpha$ одинаковы (это видно и в таблице).

2.3 Существование и единственность — прямое доказательство

Хотя существование и единственность получатся из теоремы Банаха (раздел 4), полезно увидеть их «руками».

$$F'(x) = 2 - \frac{4x\,e^{-2x^2}}{1 + e^{-2x^2}} = 2 - \frac{4x}{e^{2x^2} + 1} = 2\bigl(1 - G'(x)\bigr),$$

где $G'(x) = \dfrac{2x}{e^{2x^2} + 1}$ (см. ниже). Так как $|G'(x)| \leq \frac{1}{2}$ (лемма 1), имеем $F'(x) \geq 1 > 0$: F строго возрастает. Кроме того, $2x \leq F(x) \leq 2x + \ln 2$, поэтому $F(x) \to \pm\infty$ при $x \to \pm\infty$. По теореме о промежуточном значении F принимает значение $2\ln|\alpha|$ ровно один раз.

Локализация корня. Так как $0 < \ln(1 + e^{-2x^2}) \leq \ln 2$, из $x^* = \ln|\alpha| - \frac{1}{2}\ln(1 + e^{-2x^{*2}})$ следует

$$\ln|\alpha| - \tfrac{1}{2}\ln 2 \;\leq\; x^* \;<\; \ln|\alpha|.$$

Это удобная проверка ответа: например, при $\alpha = 10$ получаем $1.956 \leq x^* < 2.3026$, и действительно $x^* \approx 2.302573$ (при больших $|x|$ слагаемое e^{-2x^2} ничтожно, и $x^* \approx \ln|\alpha|$).

3 Метод простой итерации

3.1 Идея

Уравнение записывают в виде $x = G(x)$ и строят последовательность $x_{n+1} = G(x_n)$ с начальным приближением x_0. Если G *сжимает* расстояния, последовательность сходится к единственной неподвижной точке — решению.

Теорема 1 (Банаха о сжимающем отображении, для $\mathbb{R}$)**.** *Пусть $G\colon \mathbb{R} \to \mathbb{R}$ и существует $k \in [0,1)$ такое, что $|G(u)-G(v)| \le k|u-v|$ для всех u, v. Тогда у G есть единственная неподвижная точка x^*, и для любого x_0 последовательность $x_{n+1} = G(x_n)$ сходится к x^*, причём*

(а) апостериорная оценка: $|x_n - x^*| \le \dfrac{|x_{n+1} - x_n|}{1-k}$; *(б) априорная оценка:* $|x_n - x^*| \le \dfrac{k^n}{1-k}|x_1 - x_0|$.

Доказательство. По индукции $|x_{m+1} - x_m| \le k^m |x_1 - x_0|$. Для $p \ge 1$

$$|x_{n+p} - x_n| \le \sum_{m=n}^{n+p-1} |x_{m+1} - x_m| \le |x_1 - x_0| \sum_{m=n}^{\infty} k^m = \frac{k^n}{1-k}|x_1 - x_0| \xrightarrow[n\to\infty]{} 0,$$

т. е. (x_n) фундаментальна и (полнота $\mathbb{R}$) сходится к некоторому x^*. Сжатие непрерывно, поэтому, переходя к пределу в $x_{n+1} = G(x_n)$, получаем $x^* = G(x^*)$. Если $y^* = G(y^*)$ — ещё одна неподвижная точка, то $|x^* - y^*| = |G(x^*) - G(y^*)| \le k|x^* - y^*|$, откуда $x^* = y^*$. Устремляя $p \to \infty$ в оценке выше, получаем (б). Оценка (а):

$$|x_n - x^*| \le |x_n - x_{n+1}| + |x_{n+1} - x^*| = |x_n - x_{n+1}| + |G(x_n) - G(x^*)| \le |x_n - x_{n+1}| + k|x_n - x^*|,$$

и остаётся перенести $k|x_n - x^*|$ влево и разделить на $1 - k > 0$. □

3.2 Выбор отображения G

Из логарифмической формы $2x + \ln(1 + e^{-2x^2}) = 2\ln|\alpha|$ выражаем x:

$$\boxed{G(x) = \ln|\alpha| - \frac{1}{2}\ln\bigl(1 + e^{-2x^2}\bigr)}$$

Почему именно так? Нам нужно, чтобы $|G'|$ было меньше 1 *на всей прямой* и *при всех* α. Здесь α входит только аддитивной константой $\ln|\alpha|$, которая исчезает при дифференцировании, а оставшаяся часть $-\frac{1}{2}\ln(1 + e^{-2x^2})$ — ограниченная и очень «плоская» функция. Другие естественные варианты хуже:

- $x = x - \lambda\bigl(1 + e^{-2x^2} - \alpha^2 e^{-2x}\bigr)$: производная $1 - \lambda(\dots)$ зависит от α и неограничена (из-за e^{-2x}), одного λ на всю прямую не подобрать;
- выражать x из $e^{-2x^2} = \alpha^2 e^{-2x} - 1$: получаем $x^2 = -\frac{1}{2}\ln(\alpha^2 e^{-2x} - 1)$ — нужно следить за областью определения (аргумент логарифма должен быть положителен) и за знаком корня, удобной оценки нет.

3.3 Доказательство сжимаемости

Лемма 1. *Для всех $x \in \mathbb{R}$ и всех $\alpha \ne 0$:* $\quad |G'(x)| \le \frac{1}{2}$.

Доказательство. По цепному правилу $\bigl(\ln(1 + e^{-2x^2})\bigr)' = \dfrac{-4x\,e^{-2x^2}}{1 + e^{-2x^2}}$, поэтому

$$G'(x) = -\frac{1}{2} \cdot \frac{-4x\,e^{-2x^2}}{1 + e^{-2x^2}} = \frac{2x\,e^{-2x^2}}{1 + e^{-2x^2}} = \frac{2x}{e^{2x^2} + 1}$$

(умножили числитель и знаменатель на e^{2x^2}). Так как $e^t \geq 1 + t$, при $t = 2x^2$ знаменатель $e^{2x^2} + 1 \geq 2 + 2x^2$. Значит,

$$|G'(x)| \leq \frac{2|x|}{2(1+x^2)} = \frac{|x|}{1+x^2} \leq \frac{1}{2},$$

где последнее неравенство — это $2|x| \leq 1 + x^2 \iff (|x| - 1)^2 \geq 0$. □

По формуле Лагранжа $G(u) - G(v) = G'(\xi)(u - v)$ для некоторого ξ между u и v, поэтому $|G(u) - G(v)| \leq \frac{1}{2}|u - v|$: G — сжатие с константой $k = \frac{1}{2}$.

Замечание (точная константа). Численно $\sup_x |G'(x)| \approx 0.39295$ (достигается при $|x| \approx 0.6078$). Мы используем доказуемую оценку $k = \frac{1}{2}$: она немного грубее, зато строго обоснована; от неё зависит только критерий остановки.

3.4 Критерий остановки

По апостериорной оценке (а) с $k = \frac{1}{2}$:

$$\frac{|x_{n+1} - x_n|}{1-k} \leq \varepsilon \iff |x_{n+1} - x_n| \leq \frac{\varepsilon}{2} = 5 \cdot 10^{-13} \quad \Longrightarrow \quad |x_n - x^*| \leq \varepsilon = 10^{-12}.$$

Поэтому в ответ выдаётся x_n — приближение, для которого *гарантирована* точность ε, а n — его номер (как в образце).

Априорная оценка числа шагов. $x_0 = 0$, $x_1 = \ln|\alpha| - \frac{1}{2}\ln 2$. Из (б) точность ε гарантирована, как только $\frac{k^n}{1-k}|x_1 - x_0| \leq \varepsilon$, т. е. $n \geq \log_2 \frac{2|x_1|}{\varepsilon}$. Например, при $\alpha = 2$: $|x_1| = \frac{1}{2}\ln 2 \approx 0.347$, $n \geq \log_2(6.9 \cdot 10^{11}) \approx 39.3$. Фактически хватает 26 шагов (критерий (а) точнее, чем (б)).

3.5 Почему число итераций разное

Вблизи корня ошибка убывает примерно как $|x_{n+1} - x^*| \approx |G'(x^*)|\,|x_n - x^*|$, т. е. реальная скорость определяется числом $|G'(x^*)|$, а не грубой константой k:

$\lvert\alpha\rvert$	x^*	$G'(x^*)$	n
1	−0.302822	−0.275	21
2	0.430809	0.352	26
3	1.045342	0.211	19
4	1.375027	0.061	12
6	1.790942	0.0059	7
10	2.302573	0.00011	5

При больших $|\alpha|$ корень далеко от нуля, $G'(x^*) \approx 2x^* e^{-2x^{*2}}$ очень мал — сходимость почти мгновенная. Самый медленный случай — $|\alpha|$ около 2, когда x^* близок к точке максимума $|G'|$.

3.6 Вычислительные тонкости

- $\ln(1+t)$ при малом $t = e^{-2x^2}$ вычисляется функцией `std::log1p(t)`: обычный `log(1+t)` при $t < 10^{-16}$ дал бы $\log 1 = 0$ с потерей всех значащих цифр t.
- При больших $|x|$ величина e^{-2x^2} становится машинным нулём (*underflow*) — это безопасно: тогда $G(x) = \ln|\alpha|$, что верно с машинной точностью.
- $\ln|\alpha|$ вычисляется *один раз* до цикла.
- Критерий $|x_{n+1} - x_n| \leq 5 \cdot 10^{-13}$ достижим в арифметике `double`: ошибка округления при вычислении G порядка $10^{-16}\max(1, |x|)$, а для любых конечных `double` $\alpha \neq 0$ корень удовлетворяет $|x^*| \leq |\ln|\alpha|| + 1 < 746$. На всякий случай в коде есть страховочный лимит `MAX_ITER`.

4 Разбор кода

```cpp
#include <cmath>
#include <cstdlib>
#include <iomanip>
#include <iostream>

// Уравнение: 1 + exp(-2x^2) = alpha^2 * exp(-2x).
// Эквивалентная форма неподвижной точки (alpha != 0):
//     x = G(x) = ln|alpha| - ln(1 + exp(-2x^2)) / 2,
// |G'(x)| <= K = 1/2 на всей прямой.

const double EPS = 1e-12;
const double K = 0.5;
const int MAX_ITER = 10000;

double G(double x, double log_abs_alpha) {
    return log_abs_alpha - 0.5 * std::log1p(std::exp(-2.0 * x * x));
}

void solve(double alpha) {
    std::cout << std::defaultfloat << std::setprecision(12)
              << std::setw(14) << alpha;

    if (alpha == 0.0) {
        std::cout << std::setw(21) << "no solution"
                  << std::setw(8) << "-" << '\n';
        return;
    }

    const double log_abs_alpha = std::log(std::abs(alpha));
    double x = 0.0;
    int n = 0;

    while (true) {
        double new_x = G(x, log_abs_alpha);
        double step = std::abs(new_x - x);

        if (step / (1.0 - K) <= EPS) {
            break;
        }

        x = new_x;
        ++n;

        if (n > MAX_ITER) {
            std::cerr << "iteration limit exceeded for alpha = " << alpha << '\n';
            break;
        }
    }

    std::cout << std::fixed << std::setprecision(15)
              << std::setw(21) << x
              << std::setw(8) << n << '\n';
}

void print_header() {
    std::cout << std::setw(14) << "alpha"
              << std::setw(21) << "x"
              << std::setw(8) << "n" << '\n';
```

```
59 }
60
61 int main(int argc, char* argv[]) {
62     if (argc == 2) {
63         char* end = nullptr;
64         double alpha = std::strtod(argv[1], &end);
65
66         if (end == argv[1] || *end != '\0' || !std::isfinite(alpha)) {
67             std::cerr << "alpha must be a number\n";
68             return 1;
69         }
70
71         print_header();
72         solve(alpha);
73         return 0;
74     }
75
76     if (argc == 3) {
77         char* end1 = nullptr;
78         char* end2 = nullptr;
79         long first = std::strtol(argv[1], &end1, 10);
80         long last = std::strtol(argv[2], &end2, 10);
81
82         if (end1 == argv[1] || end2 == argv[2] ||
83             *end1 != '\0' || *end2 != '\0' || first >= last) {
84             std::cerr << "bounds must be integers and first < last\n";
85             return 1;
86         }
87
88         print_header();
89         for (long alpha = first; alpha <= last; ++alpha) {
90             solve(static_cast<double>(alpha));
91         }
92         return 0;
93     }
94
95     std::cerr << "Usage: iterative_solver alpha\n"
96               << "   or: iterative_solver first_integer last_integer\n";
97     return 1;
98 }
```

Строки 1–4. Подключаем `cmath` (`exp`, `log`, `log1p`, `abs`, `isfinite`), `cstdlib` (`strtod`, `strtol` — разбор аргументов), `iomanip` (форматирование таблицы), `iostream` (вывод).

Строки 11–13. Константы: требуемая точность $\varepsilon = 10^{-12}$, константа сжатия $k = \frac{1}{2}$ (лемма 1) и страховочный лимит итераций.

Строки 15–17. Функция `G` — отображение $G(x) = \ln|\alpha| - \frac{1}{2}\ln(1 + e^{-2x^2})$. Параметр передаётся уже в виде $\ln|\alpha|$.

Строки 19–27. Функция `solve(alpha)` сначала печатает α; при $\alpha = 0$ сообщает, что решений нет (раздел 2.1).

Строки 29–31. Вычисляем $\ln|\alpha|$, начальное приближение $x_0 = 0$, счётчик $n = 0$.

Строки 33–48. Цикл метода простой итерации. На каждом шаге `new_x` $= x_{n+1} = G(x_n)$, `step` $= |x_{n+1} - x_n|$. Если `step`$/(1-k) \le \varepsilon$, то по апостериорной оценке $|x_n - x^*| \le \varepsilon$ — выходим, *не* заменяя x на `new_x`, так что в `x` лежит x_n, а в `n` — его номер. Иначе переходим к следующему приближению.

Строки 50–52. Печать x_n с 15 знаками после точки и номера n.

Строки 55–59. Заголовок таблицы.

Функция `main`. Два режима, как в образце.

- Один аргумент — вещественное α. `strtod` разбирает строку; проверяем, что строка — целиком число (`end` указывает на конец строки) и что оно конечно (`isfinite`).
- Два аргумента — целые $a < b$; решаем уравнение для всех целых $\alpha \in [a, b]$, включая концы.
- Иначе — подсказка по использованию и код возврата 1.

Отличия от образца. Образец решает *систему* из двух уравнений (вектор (x, y), норма $\|\cdot\|_1$, $k = \frac{3}{4}$); у нас *одно* уравнение, поэтому одна переменная x, модуль вместо нормы и $k = \frac{1}{2}$. Добавлена обработка $\alpha = 0$ (у образца решение есть при всех α, у нас — нет) и страховочный лимит итераций. Структура программы, формат вывода и интерфейс командной строки — те же.

5 Проверка результатов

Корни были независимо перепроверены в Python с библиотекой `mpmath` (40 значащих цифр, метод Ньютона `findroot`) для всех целых α от -10 до 10, $\alpha \neq 0$. Фактическая ошибка $|x_n - x^*|$ всюду не превосходит $4.5 \cdot 10^{-13} < 10^{-12}$ (наибольшая — при $|\alpha| = 8$), а невязка $1 + e^{-2x_n^2} - \alpha^2 e^{-2x_n}$ по модулю не больше $1.4 \cdot 10^{-12}$. Значения $\pm\alpha$ дают одинаковые ответы, все корни лежат в отрезке $[\ln|\alpha| - \frac{1}{2}\ln 2,\ \ln|\alpha|]$ из раздела 2.3.

Дополнительно проверены крайние случаи: $\alpha = 10^{300}$ ($x \approx 690.7755$, 2 итерации), $\alpha = 10^{-300}$ ($x \approx -690.7755$), $\alpha = 0.5$ ($x \approx -0.81175$, 27 итераций), некорректный ввод (`abc`) — выдаётся сообщение об ошибке.

6 Возможные вопросы на сдаче

Почему метод сходится при любом α? — Производная G' не зависит от α и по модулю $\leq \frac{1}{2}$ на всей прямой; по теореме Банаха итерации сходятся из любой начальной точки.
Почему решение единственно? — Неподвижная точка сжатия единственна (или: $F'(x) \geq 1 > 0$, функция F строго возрастает).
Что будет при $\alpha = 0$? — Решений нет: левая часть ≥ 1, правая равна 0.
Откуда критерий остановки? — Из апостериорной оценки $|x_n - x^*| \leq \frac{|x_{n+1} - x_n|}{1-k}$; при $k = \frac{1}{2}$ это $|x_{n+1} - x_n| \leq 5 \cdot 10^{-13}$.
Почему печатается x_n, а не x_{n+1}? — Именно для x_n гарантирована оценка $\leq \varepsilon$ (так же сделано в образце). На самом деле x_{n+1} ещё точнее: $|x_{n+1} - x^*| \leq \frac{k}{1-k}|x_{n+1} - x_n|$.
Почему при $|\alpha| = 2$ итераций больше всего? — Скорость определяется $|G'(x^*)|$; при $|\alpha| \approx 2$ корень $x^* \approx 0.43$ близок к точке максимума $|G'|$ (≈ 0.61), и $|G'(x^*)| \approx 0.35$ — наибольшее среди целых α.
Зачем `log1p`? — Для точного вычисления $\ln(1 + t)$ при очень малых t.
Можно ли взять $k \approx 0.393$? — Да, это точная верхняя грань $|G'|$, но её надо обосновать (найдена численно); $k = \frac{1}{2}$ доказывается в одну строку, а на ответ и гарантию точности это не влияет.